

Exam 22-02-2026 - Selected Solutions

Problem 2: Optimal Line Breaking for Justified Text

1. Formulazione Ricorsiva (Point 1)

L'obiettivo è trovare una sequenza di tagli che minimizzi la *badness* totale, includendo anche l'ultima riga [cite: 5]. Definiamo $OPT[j]$ come la minima *badness* totale per formattare in modo ottimo il prefisso di parole $w_1, \dots, w_j$.

Per calcolare $OPT[j]$, decidiamo da quale parola w_i far iniziare l'**ultima riga** di questo prefisso (con $1 \leq i \leq j$). Se l'ultima riga contiene le parole da i a j , il costo di questa scelta è dato dalla *badness* della riga stessa, $B(i, j)$, sommata al costo ottimo per formattare tutte le parole precedenti, $OPT[i - 1]$ [cite: 5].

Provando tutti i possibili punti di inizio i e prendendo il minimo, otteniamo:

$$OPT[j] = \min_{1 \leq i \leq j} OPT[i - 1] + B(i, j)$$

Caso base:

$$OPT[0] = 0$$

2. Algoritmo di Programmazione Dinamica e Complessità (Point 2)

Precalcoliamo le lunghezze usando somme prefisse $P[k] = \sum_{x=1}^k l_x$, in modo da calcolare la lunghezza occupata tra i e j in tempo $O(1)$ [cite: 5].

Pseudocodice:

```

Algorithm OptimalLineBreaking(L, l_1...l_n):
    P[0] = 0
    For k = 1 to n:
        P[k] = P[k-1] + l_k

    OPT[0] = 0
    Break[0] = 0

    For j = 1 to n:
        OPT[j] = +inf
        For i = 1 to j:
            slack = L - (j - i) - (P[j] - P[i-1])
            if slack >= 0:
                badness = slack^3
                if OPT[i-1] + badness < OPT[j]:
                    OPT[j] = OPT[i-1] + badness
                    Break[j] = i

    return OPT[n], Break

```

Complessità: * Tempo: Precalcolo di P in $O(n)$ [cite: 5]. I cicli annidati eseguono $\sum_{j=1}^n j = O(n^2)$ passi con corpo interno in $O(1)$ [cite: 5]. Tempo totale: $O(n^2)$ [cite: 5]. * **Spazio:** Vettori di dimensione $n + 1$ per OPT , P e $Break$ [cite: 5]. Spazio totale: $O(n)$ [cite: 5].

3. Ricostruzione della Soluzione (Point 3)

Partiamo dall'ultima parola n e risaliamo a ritroso usando `Break[j]` [cite: 5]:

```

Algorithm ReconstructSolution(Break, n):
    Lines = empty list
    curr = n
    While curr > 0:
        start_word = Break[curr]
        Lines.push( (start_word, curr) )
        curr = start_word - 1
    Reverse(Lines)
    Return Lines

```

Il ciclo esegue al più n iterazioni, portando il tempo di ricostruzione a $O(n)$ [cite: 5].

Problem 4: NP-completeness of 3-Dimensional Matching and Subset Sum

1. Problem Definitions (Point 1)

- **3-Dimensional Matching (3DM):**
 - **Input:** Tre insiemi disgiunti $X = x_1, \dots, x_n, Y = y_1, \dots, y_n, Z = z_1, \dots, z_n$ di cardinalità n , e una collezione di triple $T \subseteq X \times Y \times Z$ [cite: 6].
 - **Domanda:** Esiste un sottoinsieme $S \subseteq T$ di cardinalità $|S| = n$ tale che ogni elemento di $X \cup Y \cup Z$ compaia in esattamente una tripla in S [cite: 6]?
 - **Subset Sum:**
 - **Input:** Un insieme di interi positivi $W = w_1, \dots, w_m$ e un intero target T [cite: 6].
 - **Domanda:** Esiste un sottoinsieme $S \subseteq 1, \dots, m$ tale che $\sum_{i \in S} w_i = T$ [cite: 6]?
-

2. Subset Sum è NP-Completo (Riduzione da 3DM) (Point 2)

Appartenenza a NP: Dato un sottoinsieme di indici come certificato, possiamo sommare gli interi corrispondenti e verificare che la somma eguagli T in tempo polinomiale $O(m)$.

Riduzione da 3DM: Sia data un'istanza di 3DM con insiemi X, Y, Z di taglia n e triple $T = t_1, \dots, t_m$. L'universo totale ha $3n$ elementi: $x_1, \dots, x_n, y_1, \dots, y_n, z_1, \dots, z_n$.

- **Scelta della Base:** Fissiamo una base numerica $B = m + 1$. Scegliere una base strettamente maggiore del numero totale di elementi da sommare (m) impedisce fisicamente il verificarsi di qualsiasi riporto aritmetico tra posizioni contigue.
- **Costruzione degli Interi:** Per ogni tripla $t_r = (x_i, y_j, z_k) \in T$, creiamo un intero w_r la cui espressione in base B è un vettore di $3n$ cifre con valore 1 nelle posizioni associate a x_i, y_j, z_k e 0 altrove:

$$w_r = B^{3n-i} + B^{2n-j} + B^{n-k}$$

- **Definizione del Target:** Il target T_{target} è il numero che in base B ha esattamente $3n$ cifre tutte pari a 1:

$$T_{target} = \sum_{p=0}^{3n-1} B^p = \underbrace{11 \dots 11}_{3n \text{ volte}} \text{ in base } B$$

Correttezza ($\iff$): * ($\implies$) Se esiste un 3DM $S \subseteq T$ di dimensione n , ogni elemento compare esattamente una volta. Sommando i numeri $w_r \mid t_r \in S$, in ciascuna delle $3n$ cifre la somma è esattamente 1. Non essendoci riporti, la somma totale equivale esattamente a T_{target} . * ($\impliedby$) Supponiamo che esista un sottoinsieme che sommi a T_{target} . Poiché $B > m$, nessuna somma su una singola colonna può generare riporto alla colonna successiva. Quindi la somma deve fare 1 in ogni singola colonna: ogni elemento x_i, y_j, z_k compare in esattamente una tripla selezionata. Le triple scelte formano un 3DM perfetto.

3. 3-Dimensional Matching è NP-Completo (Riduzione da 3-SAT) (Point 3)

Appartenenza a NP: Dato un sottoinsieme di triple S , un verificatore in tempo $O(|S| \cdot n)$ controlla che $|S| = n$ e che gli insiemi siano una partizione esatta di $X \cup Y \cup Z$.

Riduzione da 3-SAT: Sia data una formula in forma 3-CNF con n variabili $x_1, \dots, x_n$ ed m clausole $C_1, \dots, C_m$.

Costruiamo un'istanza di 3DM con tre insiemi disgiunti di elementi X, Y, Z :

- Gadget di Variabile (Ciclo Alternante):** Per ogni variabile x_i , introduciamo m elementi interni $a_{i,1}, \dots, a_{i,m} \in X$ e m elementi interni $b_{i,1}, \dots, b_{i,m} \in Y$. Creiamo poi $2m$ elementi di appoggio: $t_{i,1}, \dots, t_{i,m} \in Z$ e $f_{i,1}, \dots, f_{i,m} \in Z$. Colleghiamo questi vertici con $2m$ triple a cerchio: * Triple False: $(a_{i,j}, b_{i,j}, f_{i,j})$ per ogni $j \in 1, \dots, m$ * Triple True: $(a_{i,j}, b_{i,j+1}, t_{i,j})$ per ogni $j \in 1, \dots, m-1$ e $(a_{i,m}, b_{i,1}, t_{i,m})$ per chiudere l'anello. * *Proprietà:* Poiché gli elementi $a_{i,j}$ e $b_{i,j}$ appartengono solo a questo gadget, per coprirli tutti senza sovrapposizioni siamo obbligati a scegliere o **tutte le triple True** (lasciando liberi tutti gli elementi $f_{i,j}$ per il resto del grafo) o **tutte le triple False** (lasciando liberi tutti i $t_{i,j}$).
- Gadget di Clausola:** Per ogni clausola C_j , introduciamo due elementi $p_j \in X$ e $q_j \in Y$. Per ciascuno dei tre letterali presenti in C_j : * Se il letterale è x_i , aggiungiamo la tripla $(p_j, q_j, t_{i,j})$. * Se il letterale è $\neg x_i$, aggiungiamo la tripla $(p_j, q_j, f_{i,j})$. * *Proprietà:* Per coprire la coppia della clausola p_j, q_j , dobbiamo consumare un elemento Z associato a un suo letterale soddisfatto. Questo è possibile solo se tale elemento non è già stato assorbito dal ciclo della variabile (il letterale deve essere vero).
- Gadget di Pulizia (Garbage Collection):** In ogni gadget di variabile vengono lasciati liberi m elementi in Z , ma ogni clausola ne assorbe solo 1. Rimangono complessivamente $n \cdot m - m = (n-1)m$ elementi in Z non accoppiati. Introduciamo $(n-1)m$ coppie ausiliarie (g_k^X, g_k^Y) con $g_k^X \in X$ e $g_k^Y \in Y$, e permettiamo loro di formare una tripla con qualsiasi elemento $t_{i,j}$ o $f_{i,j}$ di Z .

Correttezza ($\iff$): * ($\implies$) Se la formula è soddisfacibile, scegliamo le triple True/False coerenti con l'assegnazione nei gadget di variabile. Poiché ogni clausola ha almeno un letterale vero, per ogni clausola C_j copriamo p_j, q_j con l'elemento Z di tale letterale. Gli elementi Z rimasti liberi vengono infine saturati in modo biiettivo dai gadget di pulizia, formando un matching perfetto. * ($\impliedby$) Se esiste un 3DM, la scelta delle triple nei cicli di variabile impone un'assegnazione di verità coerente (non contraddittoria). Poiché ogni coppia di clausola p_j, q_j deve essere coperta, deve consumare un elemento Z rimasto disponibile, garantendo che ogni clausola abbia almeno un letterale vero. La formula è soddisfacibile.